

Conversation Memory and Query Rewriting Notes

History, buffers, summaries, structured state, follow-up rewriting, sessions, and persistence.

Engineering notes and worked references

Contents

1. Why conversation memory lives in the application
2. Raw history
3. Recent-message buffer
4. Summary memory
5. Structured state
6. Vague follow-ups
7. Topic changes
8. Session isolation
9. Persistence
10. Debugging memory problems

1. Why conversation memory lives in the application

An API model does not automatically retain the application's prior requests. If a later answer needs earlier context, the application has to store and resend or summarize that context.

This is why session storage and model context are related but not the same thing. The database can keep the whole conversation even when only part of it is sent to the model.

Example from use

Store messages by session_id, then select the relevant recent messages for the next request.

A process restart should not erase conversation history if persistence is required.

Things people mix up

- Assuming the provider keeps your previous call.
- Keeping one global Python list for every user.
- Sending the full database history on every request.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For why conversation memory lives in the application, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

2. Raw history

The simplest memory method is to resend earlier user and assistant messages in order. It preserves exact wording and is easy to understand.

The downside is context growth. Long conversations become slower and can crowd out retrieved evidence or hit the model context limit.

Example from use

Raw history works well for a short five-message support conversation.

It becomes less attractive for a conversation with hundreds of turns.

Things people mix up

- Dropping the system prompt when replaying history.
- Mixing messages from different session IDs.
- Assuming more history is always more useful.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For raw history, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

3. Recent-message buffer

A buffer keeps the most recent N messages or turns exactly and drops older messages from the model context.

It is predictable and preserves recent wording. It can lose older facts that the user still expects the assistant to remember.

Example from use

Keep the last 6 to 8 messages for a short intern project.

Store the full conversation separately if you need an audit trail.

Things people mix up

- Confusing 'last 8 messages' with 'last 8 user turns'.
- Deleting old messages from storage just because they are not sent to the model.
- Using a buffer when one critical old setting must never be forgotten.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For recent-message buffer, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

4. Summary memory

Summary memory compresses older conversation into a shorter note. It can preserve the gist while reducing token use.

A summary is lossy. Exact model names, IDs, numbers, or choices may disappear unless the summarizer is explicitly told to retain them or they are stored separately.

Example from use

A summary can remember that the user is comparing Chroma and FAISS while recent messages keep the exact question.

Store a selected project ID in structured state instead of hoping the summary preserves it.

Things people mix up

- Treating a summary as an exact transcript.
- Overwriting the only copy of history with the summary.
- Letting the summary add details that were never said.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For summary memory, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

5. Structured state

Some conversational facts fit better in normal fields than in free-text memory. Examples include session ID, selected document, saved note ID, user-selected mode, or current incident ID.

Structured state is easier to validate and query than asking the model to infer the same value from a long conversation.

Example from use

Store `current_document_id='doc-17'` next to the session.

Keep user message history and selected filters separately.

Things people mix up

- Storing every sentence as a database column.
- Using model-generated IDs without validation.
- Forgetting to update state when the user explicitly changes selection.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For structured state, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.

- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

6. Vague follow-ups

A follow-up such as 'what about top-k?' can make sense to the conversation model but still be weak for document retrieval.

Before vector search, rewrite the message into a standalone query using only the context needed to resolve the reference.

Example from use

Previous topic: 'How does vector search choose results?' Follow-up: 'what about top-k?' Rewrite: 'How does top-k affect vector search result selection?'.

Previous topic: 'Tell me about RAG.' Follow-up: 'and chunk overlap?' Rewrite: 'How does chunk overlap affect RAG retrieval?'.

Things people mix up

- Sending the vague message directly to the vector store and blaming embeddings.
- Writing a huge rewrite that contains the whole conversation.
- Changing the meaning while adding context.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For vague follow-ups, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

7. Topic changes

Not every short message is a follow-up. The user may change topic completely. A good rewriter should leave a clear new topic alone.

This is an important boundary case because forcing old context into a new question can make retrieval worse.

Example from use

After discussing memory, 'show me a Python for-loop' should stay a Python request.

After discussing top-k, 'what is the weather?' should not become a vector-search question.

Things people mix up

- Treating every pronoun-free sentence as a follow-up.
- Appending the previous topic to all queries.
- Rewriting greetings into technical searches.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For topic changes, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

8. Session isolation

Every stored message and saved state item should belong to a session or user boundary. One person's history should not appear in another person's context.

For a small project, a `session_id` column is enough to demonstrate the principle. Application code should always filter by it.

Example from use

Session research-001 can discuss embeddings while research-002 discusses FastAPI without state leaking between them.

Test with two sessions interleaved.

Things people mix up

- Using one module-level history list.
- Forgetting the `WHERE session_id` clause.

- Trusting a model to remember which rows belong to which session.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For session isolation, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

9. Persistence

In-memory memory disappears when the process restarts. SQLite provides a simple way to keep sessions, messages, and saved notes on disk.

Persistence should be tested by saving something, restarting the API, and loading it again.

Example from use

A messages table can store id, session_id, role, content, created_at.

A saved_notes table can store note_id, session_id, title, content, source_refs.

Things people mix up

- Claiming persistence without restarting the app once.
- Using a relative database path without knowing where the process writes it.
- Keeping database connections open incorrectly across requests.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For persistence, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.

10. Debugging memory problems

Memory bugs often appear as wrong context, missing context, or context leakage. The fix starts by printing or inspecting what the application actually loaded for the session.

Separate storage bugs from prompt-context bugs. The database may have the right messages while the request builder sends the wrong subset.

Example from use

Check the SQL rows for the session, then inspect the final messages array.

For a bad rewrite, log original question, history slice, and rewritten query.

Things people mix up

- Debugging from the final answer only.
- Deleting the database whenever a session looks wrong.
- Assuming a hallucinated detail proves cross-session leakage without checking stored rows.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

State to inspect

For debugging memory problems, the important question is usually 'what did the application actually load for this session?' Inspect the database rows and the final messages array instead of assuming the model saw what you expected.

Conversation systems often have several versions of state at once: full stored history, recent exact messages, a summary, selected structured values, and a rewritten retrieval query. Keeping those roles separate makes bugs easier to reason about.

Simple test cases

- Two sessions asking unrelated questions at the same time.
- A follow-up that depends on the previous turn.
- A clear topic change that should not inherit old context.
- A process restart followed by loading the same session.

If the wrong context appears, identify whether it was stored incorrectly, selected incorrectly, summarized incorrectly, or added to the model request incorrectly. Those are different bugs.